

RemedAI

Local Multi-Agent Tender Qualification System for Environmental Remediation Projects

CTSE Assignment 2 Report

Replace team member placeholders and repository URL before final submission.

1. Problem Domain

RemedAI targets a non-generic but highly practical workflow: screening environmental remediation tenders before a bid team commits scarce proposal effort. The system is designed for firms handling hazardous-site cleanup, groundwater monitoring, and telemetry-linked field delivery. These tenders are document-heavy, schedule-sensitive, and operationally risky, which makes them a strong fit for an agentic workflow rather than a conversational assistant.

The chosen sample case concerns an emergency remediation and groundwater monitoring contract. The MAS must determine whether the firm can comply with mandatory criteria, identify evidence gaps, assess schedule and capacity risk, and create a clear bid/no-bid recommendation with a practical submission plan.

System Workflow

Orchestrator	Intake	Compliance	Risk	Planner
--------------	--------	------------	------	---------

Each agent uses at least one custom Python tool, while LangGraph passes shared state and checkpoints between nodes.

Agents and Tools

Agent	Role	Primary tool	Primary output
Orchestrator	Registers case and decides whether to proceed	Case Registry Tool	Workflow decision
Intake	Extracts tender facts and requirements	Document Ingestion Tool	Tender summary
Compliance	Matches requirements against local evidence	Capability Lookup Tool	Compliance matrix
Risk	Scores execution and	Risk Scoring Tool	Risk register +

	bid risk		recommendation
Planner	Builds action plan and exports artifacts	Plan & Export Tool	Submission plan + files

2. Multi-Agent Architecture and Orchestration

The workflow is implemented with LangGraph as a stateful directed graph. The Orchestrator Agent creates the case workspace and returns a routing decision. If the case is viable, the system proceeds sequentially through the Intake, Compliance, Risk, and Planner agents. This design intentionally keeps responsibilities distinct: extraction, evidence matching, risk interpretation, and planning are separated to reduce hallucination and make agent outputs easier to test.

LangChain is used for prompt construction, output parsing, and local Ollama chat-model execution. Each agent uses a small structured-output wrapper built around a local ChatOllama model. The agents are configured with strict prompts that emphasize evidence-only reasoning, explicit ambiguity handling, and concise JSON outputs suitable for downstream state handoffs.

3. State Management

A shared TenderMASState TypedDict stores the case id, input paths, parsed tender summary, compliance assessment, risk assessment, final plan, generated artifacts, and append-only audit trail. LangGraph reducers are used for messages and audit events, preventing accidental state loss across node transitions. The graph is compiled with a SQLite SqliteSaver checkpointer so each super-step can be recovered, inspected, and resumed if needed.

4. Tooling and Observability

Each agent is paired with a domain-relevant Python tool. The Intake agent reads local files and extracts structured fields from tender documents. The Compliance agent queries a local SQLite capability database. The Risk agent applies deterministic schedule, gap, and capacity rules. The Planner agent builds backward-planned tasks and writes output artifacts. The Orchestrator uses a case registry tool to persist run metadata. All tools use type hints, docstrings, and explicit error surfaces.

For observability, the project uses two layers. First, LangSmith tracing is enabled through environment variables, allowing each chain and tool call to appear in a trace tree. Second, a local JSONL audit log is written inside each case workspace. This dual approach ensures the observability requirement remains demonstrable even when cloud tracing is unavailable during offline marking.

5. Evaluation Methodology

The repository includes a unified evaluation harness implemented with pytest. Tool-level tests validate deterministic parsing and scoring logic. A graph smoke test validates the full workflow on the seeded tender. Property-based tests probe parser robustness and risk-rule stability across varying conditions. Separate per-agent tests check the most important behavioral constraints: the Intake agent must not invent requirements, the Compliance agent must surface gaps, the Risk agent must escalate unsupported mandatory requirements, and the Planner agent must produce actionable tasks and artifact files.

This testing strategy was chosen because local SLM outputs can vary. Instead of relying only on exact text matches, the evaluation harness checks structured invariants: required fields must exist, unsupported mandatory requirements must be escalated, and all generated plans must include named owners and due dates. These assertions make the system more reliable under model drift while still allowing natural-language flexibility.

6. Deployment and Local Execution

The deployment model is intentionally simple so the markers can run it on a student machine without paid infrastructure. Ollama hosts the local model, FastAPI exposes the backend workflow, and Streamlit provides a demonstration UI. Runtime artifacts, LangGraph checkpoints, and output bundles are written to local folders. Because the assignment prohibits paid provider keys, all core reasoning is performed locally with an Ollama model and local tools.

The recommended default model is qwen3:8b because it offers strong agent-oriented behavior and tool-use capability for a small local footprint. However, the `.env` file allows the team to switch to llama3.1:8b or another Ollama-compatible model if lab hardware constraints require it. LangSmith tracing is optional at runtime; when not configured, the local JSONL audit trail still records each agent handoff and tool output.

7. Results on the Seeded Case

On the included remediation tender, the system should return a conditional bid recommendation. The company has strong evidence for remediation delivery, telemetry integration, chain-of-custody controls, and mobilization speed. However, the workflow correctly identifies a recoverable gap in 24x7 response coverage and a partner-dependent gap in drone mapping support. The resulting plan prioritizes sponsor approval, evidence pack assembly, and written partner commitments before drafting the final technical proposal.

8. Limitations and Future Work

This version focuses on a seeded local knowledge base and a structured sample tender so the workflow is easy to reproduce. In a production extension, the system should add deeper document parsing for scanned PDFs, stronger semantic retrieval over historical evidence, and an

interrupt/resume review step for procurement managers before the final recommendation is accepted.

Another valuable extension would be adding subcontractor-market discovery and supplier risk tracking, which would help convert conditional bids into evidence-backed partner strategies instead of static warnings. These additions were left out to keep the submission reliable, local, and easy to demonstrate within the assignment constraints.

9. Team Contribution Template

Member	Agent owned	Tool owned	Challenges faced
Student 1	Orchestrator	Case Registry Tool	Replace with real contribution note
Student 2	Intake	Document Ingestion Tool	Replace with real contribution note
Student 3	Compliance	Capability Lookup Tool	Replace with real contribution note
Student 4	Risk	Risk Scoring Tool	Replace with real contribution note

Repository link: [REPLACE_WITH_GITHUB_OR_GITLAB_URL](#)